Roteiro do vídeo

Sistemas Operacionais - Atividade 3 - UFS 2026.2

Duração-alvo: 15 minutos como proposta de gravação; o enunciado não fixa uma duração máxima para o vídeo.

Texto em FALA orienta a locução; EM TELA orienta o operador. Os tempos são estimativas para ensaio. Cada slide tem um apresentador, identificado no cabeçalho, e cada integrante conclui seu bloco antes de passar a palavra.

Sequência dos 14 slides

Slide e assunto	Quem fala	Janela
01. Virtualização, namespaces e cgroups	Bruno	00:00-00:35
02. Objetivo e percurso do laboratório	Bruno	00:35-01:45
03. Ambiente e kernel compartilhado	Iasmin	01:45-03:00
04. Protocolo e controle das execuções	Iasmin	03:00-03:55
05. Namespaces: visões de processos, mounts e rede	Pedro	03:55-04:50
06. Evidências de namespaces e PIDs	Pedro	04:50-05:55
07. Carga determinística e checksum	Christian	05:55-07:00
08. Auditoria e demonstração dos registros	Christian	07:00-08:30
09. Limites efetivos do cgroup v2	Marcos	08:30-09:35
10. Throttling durante a carga	Marcos	09:35-10:40
11. Seis medições de tempo decorrido	Sibele	10:40-12:05
12. Memória: processo e cgroup	Sibele	12:05-12:55
13. Mount read-only e menor privilégio	Silas	12:55-14:05
14. Limitações, aplicação e conclusão	Silas	14:05-15:00

Participação por integrante

Integrante	Slides	Tempo-alvo
Bruno Amancio Ferreira	1, 2	01:45
Iasmin Victoria Teixeira Barreto	3, 4	02:10
Pedro César Figueiredo Carneiro	5, 6	02:00
Christian Will Silva Santos Nunes	7, 8	02:35
Marcos Vinícius Dantas Aguiar	9, 10	02:10
Sibele Oliveira Cruz	11, 12	02:15
Silas Santos da Silva	13, 14	02:05

Base: apresentacao_atividade3.pdf, 14 slides em LaTeX Beamer. Ordem: Bruno → Iasmin → Pedro → Christian → Marcos → Sibele → Silas. As instruções da demonstração e de preparação aparecem no final deste PDF.

Slide 01 - Virtualização, namespaces e cgroups

Janela 00:00-00:35 | Duração-alvo 00:35



EM TELA - não ler em voz alta

Exibir a capa com a equipe e o nome de Bruno. No vídeo, identificar Bruno na tela.

Próxima ação

Avançar ao slide 2; Bruno continua.

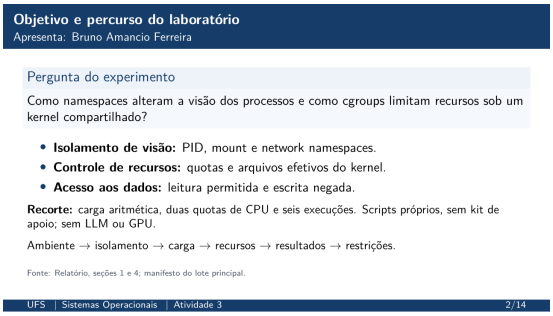
FALA - Bruno Amancio Ferreira (00:00-00:35)

Olá, eu sou Bruno, da equipe GenResearch. Este vídeo apresenta a Atividade 3 de Sistemas Operacionais da Universidade Federal de Sergipe. Construímos um laboratório próprio, sem o kit de apoio, para observar como contêineres Linux compartilham um kernel e, ao mesmo tempo, possuem visões e limites diferentes.

A apresentação segue uma sequência contínua: cada integrante explica sua parte e passa a palavra ao próximo. Vamos começar pela pergunta do experimento e pelo caminho das evidências.

Slide 02 - Objetivo e percurso do laboratório

Janela 00:35-01:45 | Duração-alvo 01:10



EM TELA - não ler em voz alta

Apontar a pergunta e as três evidências: namespaces, limites efetivos e escrita negada. Manter Bruno até concluir o slide.

Próxima ação

Avançar ao slide 3 e passar a palavra a Iasmin Victoria Teixeira Barreto.

FALA - Bruno Amancio Ferreira (00:35-01:45)

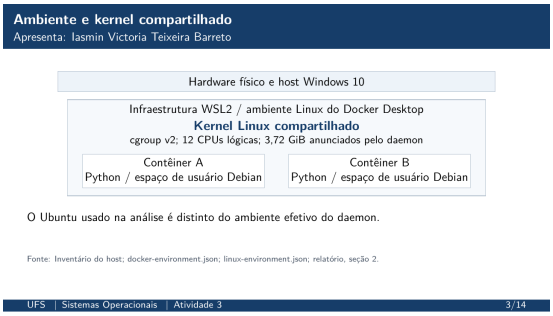
A pergunta é como namespaces mudam a visão de processos, montagens e rede, e como cgroups limitam recursos sob um kernel compartilhado. Vamos acompanhar três tipos de evidência: identificadores dos namespaces, arquivos do kernel com os limites efetivos e um teste de leitura com tentativa controlada de escrita.

Na comparação de CPU, executamos a mesma carga aritmética com quota de meia CPU e de uma CPU. São três repetições por condição, seis medições principais. O piloto é separado e não entra nesses seis resultados. Não executamos inferência de modelo ou carga de GPU.

Os registros brutos preservam valores, comandos e hashes; a análise organiza CSV, medianas e gráficos; e o auditor confere a consistência do lote. A sequência passa por ambiente, isolamento, carga, recursos, resultados e restrições. Iasmin vai mostrar as camadas do ambiente e o protocolo.

Slide 03 - Ambiente e kernel compartilhado

Janela 01:45-03:00 | Duração-alvo 01:15



EM TELA - não ler em voz alta

Iasmin assume. Percorrer o diagrama: Windows, infraestrutura WSL2, ambiente do daemon e contêineres.

Próxima ação

Avançar ao slide 4; Iasmin continua.

FALA - Iasmin Victoria Teixeira Barreto (01:45-03:00)

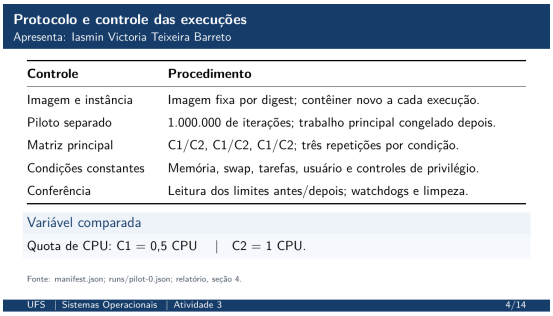
Eu sou Iasmin. Neste diagrama, a base é o hardware físico com Windows 10. Acima dele, a infraestrutura WSL2 fornece o ambiente Linux em que executa o daemon do Docker Desktop. O kernel desse ambiente é compartilhado pelos contêineres.

O daemon informou doze CPUs lógicas e aproximadamente 3,72 gibibytes de memória. O kernel observado foi 6.18.54.1, da infraestrutura WSL2, e a imagem fornece espaço de usuário Debian com Python. O fato de a imagem conter uma distribuição não significa que ela inicie um kernel próprio.

Também usamos Ubuntu WSL na análise dos arquivos. Essa distribuição de análise não deve ser confundida com o Linux efetivo do daemon. Portanto, há virtualização na infraestrutura Windows, mas os contêineres continuam sendo processos do ambiente Linux compartilhado.

Slide 04 - Protocolo e controle das execuções

Janela 03:00-03:55 | Duração-alvo 00:55



EM TELA - não ler em voz alta

Apontar a sequência C1/C2, a imagem fixa e a leitura dos limites antes/depois. Ao terminar, passar para Pedro.

Próxima ação

Avançar ao slide 5 e passar a palavra a Pedro César Figueiredo Carneiro.

FALA - Iasmin Victoria Teixeira Barreto (03:00-03:55)

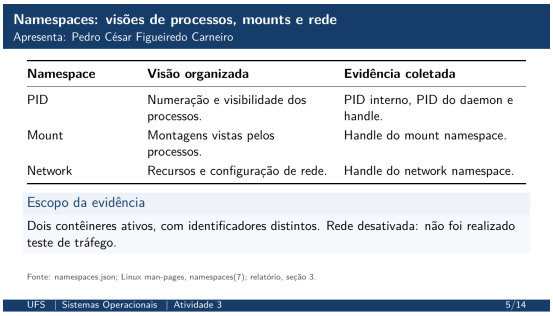
O protocolo usa a mesma imagem fixada por digest e cria um contêiner novo para cada execução. Um piloto de um milhão de iterações foi feito separadamente. Depois dele, o trabalho principal ficou congelado em vinte milhões de iterações.

A ordem é C1 e C2, repetida três vezes. C1 tem quota de meia CPU; C2, uma CPU. Memória, swap, limite de tarefas, usuário e restrições de privilégio permanecem constantes. Os arquivos do cgroup são lidos antes e depois do cálculo.

O watchdog interno é de sessenta segundos e o externo, noventa. A limpeza confirma a remoção apenas dos contêineres criados pelo laboratório. O piloto não é contado como uma das seis medições principais. Pedro assume para explicar os namespaces.

Slide 05 - Namespaces: visões de processos, mounts e rede

Janela 03:55-04:50 | Duração-alvo 00:55



EM TELA - não ler em voz alta

Pedro assume. Relacionar cada namespace à visão que ele organiza; separar identificação de namespace de teste de tráfego.

Próxima ação

Avançar ao slide 6; Pedro continua.

FALA - Pedro César Figueiredo Carneiro (03:55-04:50)

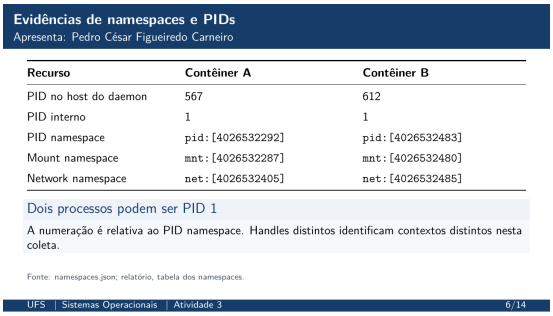
Eu sou Pedro. Vamos separar três visões. O PID namespace organiza a numeração e a visibilidade dos processos naquele contexto. O mount namespace organiza a visão das montagens. O network namespace organiza a visão de recursos de rede.

O laboratório coletou os handles desses namespaces em dois contêineres que permaneceram ativos. Esses identificadores permitem comparar os contextos: valores diferentes mostram que os processos pertencem a namespaces distintos para o recurso observado.

Essa comparação não mede isolamento absoluto. Também não foi um teste de comunicação, pois a rede dos contêineres estava desativada. A evidência é a leitura dos identificadores e dos dados dos processos. Agora vamos comparar os dois contêineres na tabela.

Slide 06 - Evidências de namespaces e PIDs

Janela 04:50-05:55 | Duração-alvo 01:05



EM TELA - não ler em voz alta

Apontar PIDs 567/612 no host do daemon, PID interno 1/1 e handles distintos. Ao terminar, passar para Christian.

Próxima ação

Avançar ao slide 7 e passar a palavra a Christian Will Silva Santos Nunes.

FALA - Pedro César Figueiredo Carneiro (04:50-05:55)

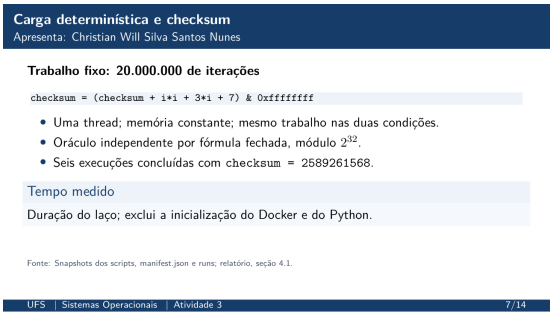
Na tabela, o host do daemon identifica os processos pelos PIDs 567 e 612. Na visão interna de cada contêiner, ambos são PID um. São numerações de contextos distintos: cada PID namespace tem sua própria visão.

Os handles PID são diferentes entre A e B; o mesmo ocorre com os handles de mount e de network. A diferença sustenta que os processos pertenciam a namespaces distintos nesses três recursos durante a coleta.

Uma nova reprodução pode produzir outros PIDs e outros identificadores. O que deve ser verificado é a relação entre as visões, não a repetição literal desses números. Também não inferimos isolamento absoluto apenas dessa tabela. Christian continua com a carga determinística e a auditoria dos registros.

Slide 07 - Carga determinística e checksum

Janela 05:55-07:00 | Duração-alvo 01:05



EM TELA - não ler em voz alta

Christian assume. Apontar N fixo, expressão do checksum e oráculo independente. Não tratar a carga como inferência de LLM.

Próxima ação

Avançar ao slide 8; Christian continua.

FALA - Christian Will Silva Santos Nunes (05:55-07:00)

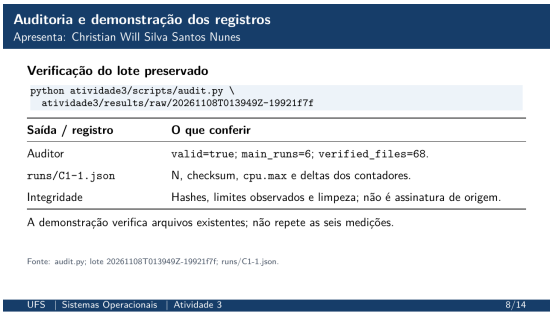
Eu sou Christian. Esta é a carga usada para comparar CPU: vinte milhões de iterações de um cálculo aritmético em Python, com uma thread e memória constante. O laço soma i ao quadrado, mais três i, mais sete, e aplica uma máscara de trinta e dois bits.

Para verificar o resultado, usamos também a fórmula fechada das somas, independente da execução do laço. Todas as seis execuções concluíram as iterações e concordaram com o checksum 2589261568, mostrado na tela.

Isso permite conferir que as duas condições realizaram o mesmo trabalho lógico. O tempo medido é o do cálculo, separado da inicialização do Docker e do Python. Esta carga não gera tokens e não utiliza LLM, GPU ou VRAM. Vamos conferir agora o lote preservado.

Slide 08 - Auditoria e demonstração dos registros

Janela 07:00-08:30 | Duração-alvo 01:30



EM TELA - não ler em voz alta

Executar o auditor do lote existente. Ampliar o terminal e mostrar valid=true, main_runs=6, verified_files=68. Depois abrir runs/C1-1.json. Se houver falha atual, explicar e abrir os registros preservados.

Próxima ação

Avançar ao slide 9 e passar a palavra a Marcos Vinícius Dantas Aguiar.

FALA - Christian Will Silva Santos Nunes (07:00-08:30)

Nesta demonstração, vamos conferir os registros de uma execução anterior. O comando mostrado verifica o lote existente; não inicia uma nova matriz de contêineres. A saída esperada do lote publicado indica valid igual a true, seis execuções principais e sessenta e oito arquivos verificados.

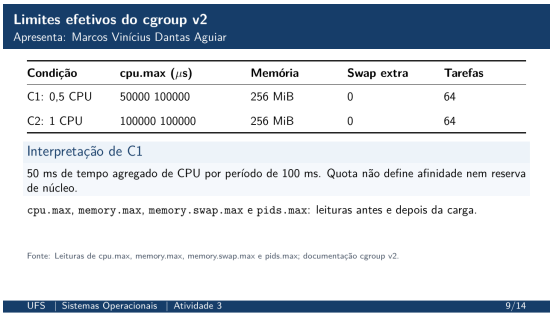
Vou abrir C1-1. O registro contém vinte milhões de iterações, checksum 2589261568 e correct igual a true. Em cpu.max, a leitura é 50000 100000. Os campos antes e depois da carga permitem conferir o limite e calcular o delta dos contadores.

No delta, nr_throttled aumentou setenta e um e throttled_usec aumentou 3544785 microssegundos. O JSON de montagem somente leitura preserva, separadamente, a escrita negada com errno trinta. Esses são registros da coleta, não acontecimentos novos em execução neste instante.

Os hashes ajudam a verificar integridade interna, mas não equivalem a uma assinatura independente da origem dos dados. Marcos assume para interpretar as quotas e os contadores do cgroup.

Slide 09 - Limites efetivos do cgroup v2

Janela 08:30-09:35 | Duração-alvo 01:05



EM TELA - não ler em voz alta

Marcos assume. Apontar cpu.max nas duas condições e os três limites mantidos constantes.

Próxima ação

Avançar ao slide 10; Marcos continua.

FALA - Marcos Vinícius Dantas Aguiar (08:30-09:35)

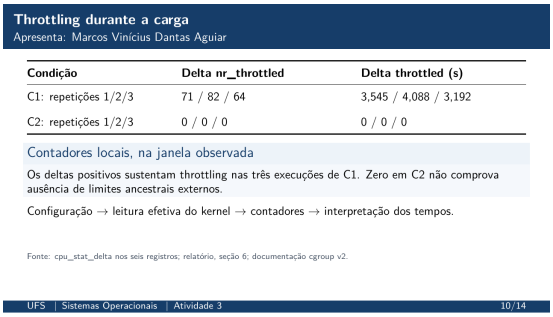
Eu sou Marcos. O limite efetivo é conferido dentro do cgroup. Em C1, cpu.max mostrou 50000 100000 microssegundos: cinquenta milissegundos de tempo agregado de CPU a cada período de cem milissegundos. Em C2, a leitura foi 100000 100000.

Quota de meia CPU não significa reservar metade de um núcleo físico, nem define afinidade. Ela limita o tempo agregado de execução no período. As opções do Docker configuram o experimento; os arquivos do kernel mostram os valores locais observados.

memory.max foi 268435456 bytes, equivalente a 256 mebibytes; memory.swap.max foi zero e pids.max, sessenta e quatro. Esses campos foram conferidos antes e depois da carga. Não provocamos OOM ou esgotamento de tarefas. Agora vamos observar o throttling.

Slide 10 - Throttling durante a carga

Janela 09:35-10:40 | Duração-alvo 01:05



EM TELA - não ler em voz alta

Apontar os deltas das três repetições. Relacionar a quota com a restrição observada; passar para Sibele ao terminar.

Próxima ação

Avançar ao slide 11 e passar a palavra a Sibele Oliveira Cruz.

FALA - Marcos Vinícius Dantas Aguiar (09:35-10:40)

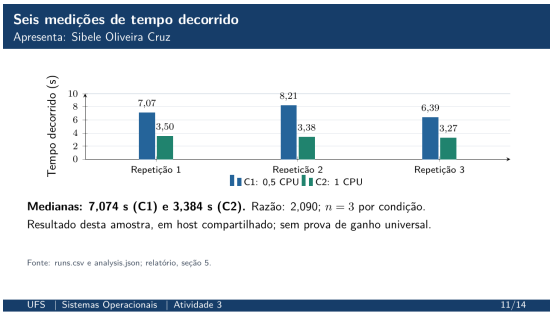
Usamos diferenças entre os contadores lidos antes e depois do cálculo. Em C1, o delta de nr_throttled foi setenta e um, oitenta e dois e sessenta e quatro nas três repetições. O tempo de throttling aumentou aproximadamente três vírgula quinhentos e quarenta e cinco, quatro vírgula zero oitenta e oito e três vírgula cento e noventa e dois segundos.

Em C2, esses deltas foram zero na janela medida. A interpretação se limita aos contadores locais disponíveis. Não significa que toda a hierarquia externa estivesse sem limites; os ancestrais externos não foram confirmados.

Temos, assim, a configuração da quota, a confirmação no kernel e o efeito registrado nos contadores. Essa combinação sustenta a interpretação do tempo decorrido, sem transformar uma amostra pequena em regra universal. Sibele continua com os resultados.

Slide 11 - Seis medições de tempo decorrido

Janela 10:40-12:05 | Duração-alvo 01:25



EM TELA - não ler em voz alta

Sibele assume. Ler o gráfico por pares, apontar as medianas e n=3. Explicar a razão entre medianas.

Próxima ação

Avançar ao slide 12; Sibele continua.

FALA - Sibele Oliveira Cruz (10:40-12:05)

Eu sou Sibele. O gráfico reúne os seis tempos do laço principal, três por condição. O primeiro conjunto corresponde à quota de meia CPU e o segundo, à de uma CPU. Cada par mantém a mesma posição de repetição para facilitar a comparação.

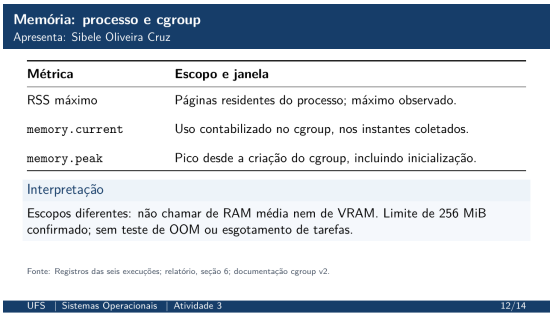
Em C1, os tempos foram aproximadamente sete vírgula zero sete, oito vírgula vinte e um e seis vírgula trinta e nove segundos. Em C2, foram três vírgula cinquenta, três vírgula trinta e oito e três vírgula vinte e sete segundos. Os rótulos do gráfico são arredondados; o CSV preserva a precisão original.

As medianas foram 7,074 e 3,384 segundos. Dividindo a mediana de C1 pela de C2, obtemos aproximadamente 2,090. Não é uma taxa de tokens por segundo e não é um ganho universal de desempenho.

Há três repetições, alternância fixa e interferência não isolada do host. Não medimos continuamente frequência e temperatura. O resultado descreve esta carga e estas condições; os contadores de throttling fornecem uma evidência adicional para sua interpretação.

Slide 12 - Memória: processo e cgroup

Janela 12:05-12:55 | Duração-alvo 00:50



EM TELA - não ler em voz alta

Distinguir RSS máximo, memory.current e memory.peak. Ao terminar, passar para Silas, que segue até o encerramento.

Próxima ação

Avançar ao slide 13 e passar a palavra a Silas Santos da Silva.

FALA - Sibele Oliveira Cruz (12:05-12:55)

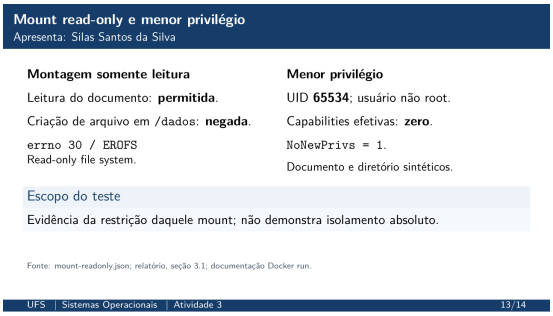
Na memória, distinguimos três campos. O RSS máximo é uma métrica do processo e descreve páginas residentes. memory.current é a contabilização do cgroup nos instantes em que foi coletado. memory.peak registra o pico desde a criação do grupo, incluindo inicialização e outros usos contabilizados.

Os escopos e as janelas são diferentes. Por isso, os valores podem divergir e não representam uma média de RAM, nem VRAM. O limite configurado foi confirmado, mas não foi feito teste de OOM ou de esgotamento de tarefas.

A próxima parte trata do acesso aos dados: Silas apresenta a montagem somente leitura, os controles de privilégio e as limitações do experimento, e faz o encerramento.

Slide 13 - Mount read-only e menor privilégio

Janela 12:55-14:05 | Duração-alvo 01:10



EM TELA - não ler em voz alta

Silas assume. Mostrar leitura permitida e escrita negada. No vídeo, abrir mount-readonly.json e apontar errno, UID e controles.

Próxima ação

Avançar ao slide 14; Silas continua.

FALA - Silas Santos da Silva (12:55-14:05)

Eu sou Silas. O laboratório criou um documento sintético e montou o diretório em modo somente leitura. Primeiro, a aplicação conseguiu ler o conteúdo. Depois, uma tentativa controlada de criar outro arquivo no mesmo diretório foi negada.

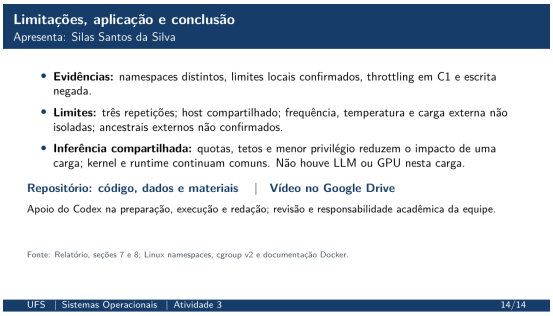
No registro mount-readonly.json, podemos conferir write_denied igual a true e errno trinta, EROFS. A informação de montagem também registra ro. O documento e o diretório são sintéticos; o teste não tenta modificar dados pessoais.

O processo executou com UID 65534, capabilities efetivas zeradas e NoNewPrivs igual a um. Esses controles limitam ações disponíveis e complementam a restrição da montagem.

A evidência é a leitura permitida e a escrita negada naquele mount, sob aquelas condições. Não concluímos segurança absoluta nem ausência de riscos do kernel ou do runtime. Vou encerrar relacionando os resultados às limitações e a ambientes de inferência compartilhados.

Slide 14 - Limitações, aplicação e conclusão

Janela 14:05-15:00 | Duração-alvo 00:55



EM TELA - não ler em voz alta

Silas permanece até o encerramento. Mostrar limitações, conclusão e links. Não retornar a apresentadores anteriores.

Próxima ação

Encerrar a gravação.

FALA - Silas Santos da Silva (14:05-15:00)

Para concluir, há três repetições por condição em um host compartilhado. Não isolamos frequência, temperatura ou carga externa, e a visão disponível não permitiu confirmar limites ancestrais externos. Essas limitações restringem a generalização dos resultados.

Em um servidor de inferência, quotas de CPU, limites de memória e tarefas, corpus somente leitura e menor privilégio ajudam a reduzir o impacto de uma carga. Kernel e runtime continuam compartilhados. Nosso cálculo não executou LLM, GPU ou carga de VRAM.

O laboratório mostrou namespaces distintos, limites locais de cgroup confirmados, throttling em C1 e escrita negada no mount observado. Com trabalho fixo, C2 apresentou menor tempo mediano nesta amostra. Os materiais estão no repositório e o link do vídeo foi registrado. Declaramos o apoio do Codex na preparação e a responsabilidade da equipe por revisar e explicar o conteúdo. Obrigado por acompanhar.

Operação da demonstração

Apoio ao slide 8. Esta página orienta Christian e a pessoa que controla a tela.

Preparar antes de apresentar ou gravar

Abrir o repositório na sua raiz. Usar um ambiente em que python execute a versão configurada. No computador desta equipe, os ensaios anteriores usaram o ambiente Python no Ubuntu WSL.

Conferir o comando antes do ensaio e deixar o terminal com fonte grande. Não instalar programas, baixar imagem ou executar novamente a matriz durante a apresentação.

Deixar dois arquivos já abertos no editor: runs/C1-1.json e mount-readonly.json, nos caminhos completos abaixo. Esses arquivos registram observações anteriores; não são eventos que estão acontecendo agora.

Comando completo em uma única linha

```
python atividade3/scripts/audit.py atividade3/results/raw/20261108T013949Z-19921f7f
```

No Bash, a quebra de linha do slide usa uma barra invertida. A versão acima pode ser colada como um único comando. O auditor só verifica o lote existente; não precisa recriar os contêineres.

O que apontar na tela

1. Saída do auditor

Apontar valid=true, main_runs=6 e verified_files=68. Se a execução atual falhar, explicar o erro e abrir os registros preservados; não afirmar sucesso atual.

2. Registro C1-1

Apontar iterations=20000000, checksum=2589261568, correct=true, cpu.max="50000 100000" e cpu_stat_delta. Em C1-1, nr_throttled aumentou 71 e throttled_usec aumentou 3544785.

3. Mount read-only

Apontar content, write_denied=true, errno=30, uid=65534 e os controles de privilégio. Usar esse arquivo principalmente no vídeo; na apresentação, respeitar o minuto reservado à demonstração.

Arquivos para deixar abertos

`atividade3/results/raw/20261108T013949Z-19921f7f/runs/C1-1.json`

`atividade3/results/raw/20261108T013949Z-19921f7f/mount-readonly.json`

Plano de contingência: se o terminal não responder, mostrar os arquivos locais e explicar que são registros da coleta. Na apresentação, passar ao slide 9 e entregar a palavra a Marcos. No vídeo, corrigir o ambiente antes de refazer o trecho, identificando a auditoria como verificação de uma coleta anterior.

Preparação da gravação

Checklist de operação. A duração de 15 minutos é uma proposta deste roteiro, a ajustar no ensaio.

Identificação de todos

Gravar cada integrante com seu rosto visível ao menos em sua entrada, seu nome completo na tela e áudio reconhecível. Os nomes estão no quadro da primeira página. Não presumir que apenas a narração sem identificação demonstra participação.

Trocas de locutor

Trocar somente ao final do segundo slide de cada integrante: 2→3, 4→5, 6→7, 8→9, 10→11 e 12→13. Cada pessoa conclui seu bloco e não retorna na exposição. Identificar quem assume e manter apenas um locutor por slide.

Slides e legibilidade

Compartilhar os slides em tela cheia e desligar notificações. Ampliar o terminal e o editor no slide 8. O público deve conseguir ler os campos que sustentam a explicação.

Falas e ritmo

Usar as falas como base, com compreensão e linguagem natural. Ensaiar o tempo total. Os números longos de handles e checksum podem ser apontados na tela; não é necessário soletrar todos os dígitos.

Encerramento

Silas apresenta os slides 13 e 14 e encerra o vídeo. Conferir se os sete integrantes foram identificados nos respectivos blocos. Os temas e intervalos estão registrados em docs/contribuicoes.md.

Demonstração honesta

Apresentar o auditor como verificação de arquivos existentes. O teste de escrita do JSON é uma evidência da coleta anterior. Não narrar essas observações como se os contêineres estivessem sendo executados naquele instante.

Revisão e publicação

Assistir ao vídeo completo, conferir o áudio de todos, os dados e a duração. Hospedar com acesso adequado ao docente. Inserir a URL no README, em VIDEO.md e na macro urlvideo do relatório LaTeX, recompilando o PDF.

Apresentação e entrega

O vídeo não substitui a apresentação de 11/11/2026. Preparar a entrega até 10/11/2026. No Classroom, disponibilizar o link do vídeo conforme o enunciado; guardar o comprovante de submissão.